

Up next in Hands-on Labs →

Lab 04: Reusable Pipeline Logic



Hands-on lab · 45 min · Env: See book overview · Overview & prereqs · Download PDF ·

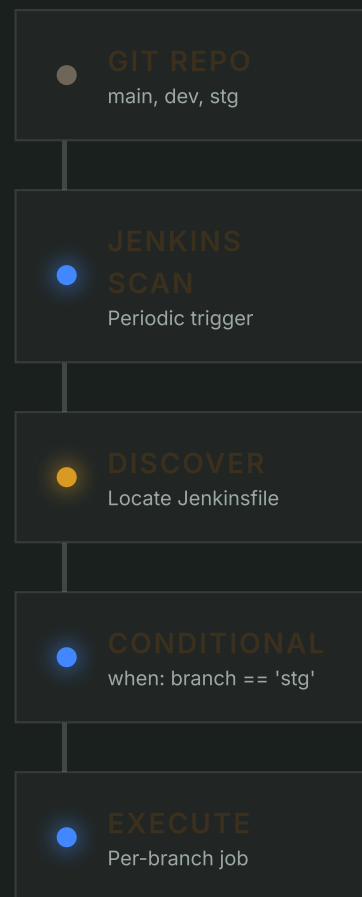
Reset checks

CI/CD ENGINEERING

Multibranch Pipelines: Automatic
Branch Discovery

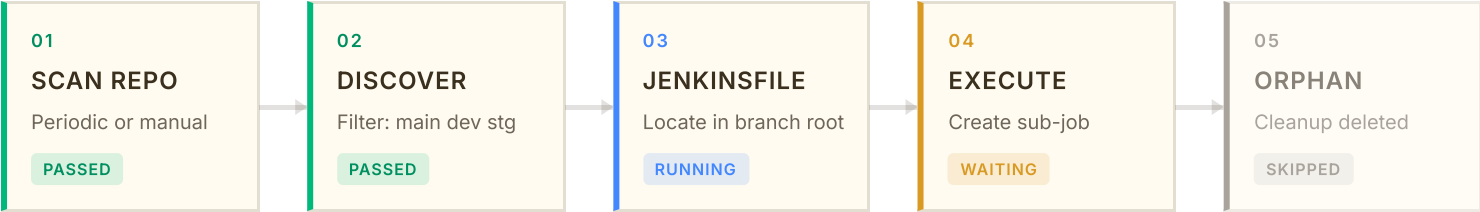
GIT INTEGRATION
BRANCH SCANNING
CONDITIONAL EXECUTION

Nabawy



01 / Multibranch Foundations

A standard pipeline executes a single predefined script. A Multibranch Pipeline is structurally different: it is a folder of individual pipeline jobs, one per discovered branch. Jenkins scans the repository, finds branches, and looks for a Jenkinsfile to build with.



WHAT

A Jenkins job type that automatically creates a sub-job for every branch in a Git repository containing a Jenkinsfile.

WHY

It eliminates the need to manually create and configure a new Jenkins job every time a developer creates a new branch.

WHEN

On a periodic schedule (e.g., every 5 minutes) or manually triggered. This task deliberately ignores webhooks to simulate an air-gapped environment.

WHAT CHANGES

The relationship between source code and CI inverts. The repository defines the jobs, not Jenkins. Branch deletion leads to job orphaning.

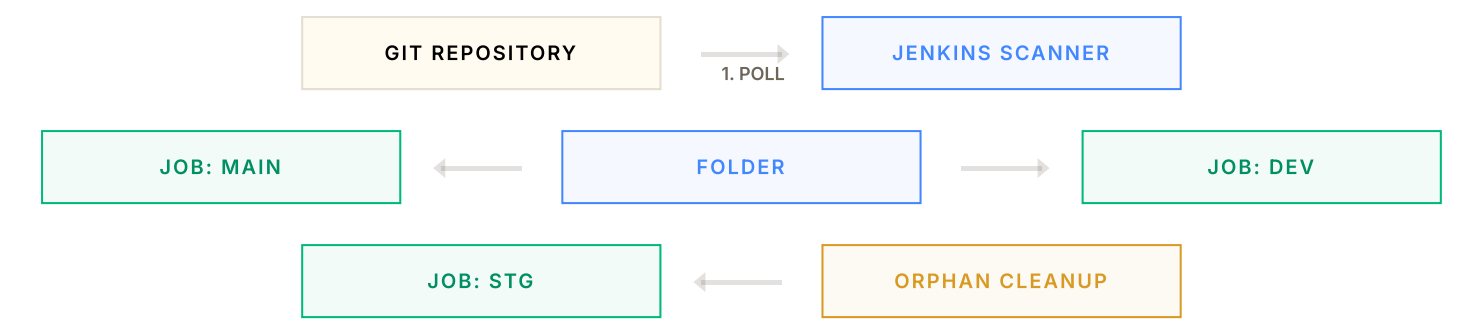
WHY THIS EXISTS

In dynamic environments with feature branches, manually managing Jenkins jobs is impossible. Multibranch pipelines push job creation down into the repository itself, making the CI configuration version-controlled and branch-specific.

02 / Architecture & Pre-Flight

The Multibranch Pipeline relies on the plain Git branch source, not GitHub-specific integration. This ensures compatibility with any Git provider and works perfectly in air-gapped or firewall-restricted networks where inbound webhooks are blocked.

EXECUTION ARCHITECTURE



PRE-FLIGHT VERIFICATION

```
$ git --version
git version 2.34.1
$ git ls-remote https://github.com/user/repo.git
e7f8a9c1d2e3f4g5h6i7j8k9l0m1n2o3 HEAD
e7f8a9c1d2e3f4g5h6i7j8k9l0m1n2o3 refs/heads/main
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6 refs/heads/dev
f1e2d3c4b5a69870123456789abcdef0 refs/heads/stg
Access confirmed.
```

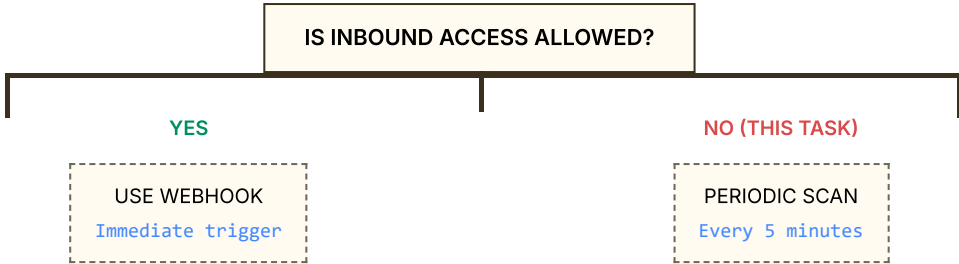
GOTCHA

Individual branch jobs inside the Multibranch folder (e.g., `main` , `dev`) are **not directly configurable**. All behavior (branch filtering, checkout strategy, etc) is controlled from the parent Multibranch Pipeline job's configuration only.

CH 03 | 03 / 08

03 / Discovery & Filtering

Configure the Multibranch Pipeline job to use a plain Git source. Filter discovery to exactly three branches: `main` , `dev` , and `stg` . Since no webhooks are used, rely on periodic scanning.



BRANCH SOURCE CONFIGURATION

```
Jenkins → New Item → Multibranch Pipeline
> Branch Source: Git (Plain Git, not GitHub)
> Project Repository: https://github.com/user/repo.git
> Behaviors → Filter by name (with wildcards)
> Include: main dev stg
> Exclude: (blank)
> Additional Behaviors:
> - Check out to matching local branch
> - Clean before/after checkout
> Scan Multibranch Pipeline Triggers: Every 5 minutes
```

GOTCHA

Ensure "Check out to matching local branch" is enabled. Without it, Jenkins checks out a detached commit. While builds may succeed, Git metadata within the pipeline won't reflect the actual branch name, breaking `when { branch 'stg' }` conditionals.

04 / Conditional Execution

A single shared Jenkinsfile must adapt its behavior based on the branch it is building. The `env.BRANCH_NAME` variable, injected by the Multibranch plugin, enables this via `when` conditionals.

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { echo 'Building ... ' } }
    stage('Staging Verification') {
      when { expression { return env.BRANCH_NAME == 'stg' } }
      steps { sh 'cat README.md' }
    }
  }
}
```

EXECUTION LANES BY BRANCH

MAIN	● Build ● Staging Verification (SKIPPED by when)
DEV	● Build ● Staging Verification (SKIPPED by when)
STG	● Build ● Staging Verification (cat README.md)

INTERVIEW CHECK

Q: How does a single Jenkinsfile adapt to different branches?

A: The Multibranch plugin injects `env.BRANCH_NAME`. The `when` directive evaluates this variable before executing a stage, allowing branch-specific logic (like deployment or extra tests) to reside safely in the same file.

05 / Lab: Orphan Cleanup

Branches are not permanent. When a branch is deleted upstream in Git, the corresponding Jenkins job must be removed to prevent clutter. This process does not happen instantly; it requires a scan.

OBSERVE	A feature branch <code>test-branch</code> was merged and deleted from the remote Git repository. The Jenkins folder still shows the job.
EXECUTE	Trigger a manual scan ("Scan Multibranch Pipeline Now") or wait for the periodic scan to execute.
VERIFY	Check the scan log. It will report <code>test-branch</code> as an orphaned item. The job will be removed from the Jenkins folder view.
PROOF	Jenkins state perfectly mirrors the Git repository state. No stale jobs remain.

FOLDER STATE TRANSITION

multibranch-job

main

dev

stg

~~test-branch~~

SCANNED

ACTIVE

ACTIVE

ACTIVE

ORPHANED

GOTCHA

Orphaned branch cleanup only happens **after a rescan**. Deleting a remote branch does not immediately remove its Jenkins job. The Jenkins folder still shows the job. **rescan** evaluates the repository state and removes orphaned items.

06 / System Map & Troubleshooting

The Multibranch Pipeline acts as an automated job manager. Understanding its scan and filter logic is critical for resolving issues where branches are missed or jobs are not created.



TROUBLESHOOTING FLOW

Branch not discovered	Filter mismatch	Check "Include" wildcard in Branch Source config	
SYMPTOM	ROOT CAUSE	RECOVERY	
when condition fails	Detached HEAD checkout	Enable "Check out to matching local branch"	
Stale jobs remain	No scan executed	Trigger manual scan or wait for periodic interval	

INTERVIEW CHECK

Q: Why use plain Git branch source instead of GitHub-specific integration?

A: The plain Git source works with any Git provider (GitLab, Bitbucket, etc.) and functions in air-gapped environments where inbound webhooks are blocked. It relies on periodic polling, prioritizing compatibility over immediate trigger speed.

Up next in Hands-on Labs →

Lab 04: Reusable Pipeline Logic

RELATED BOOKS

Lab 04: Reusable Pipeline Logic
55 min · **ADV**
→ Next in path

Lab 02: Container Delivery
50 min · **INT**
→ Previous in path

Lab 01: Automated Integration Foundations
45 min · **INT**
→ Shared tags

Lab 05: The Infrastructure Pipeline
60 min · **ADV**
→ Shared tags